

Polimorfismo (Equipos)

El polimorfismo, en este contexto, es la capacidad de tratar diferentes tipos de computadoras (como **Laptop**, **Desktop**, **Servidor**) de manera uniforme. Permite que tu sistema de registro interactúe con cualquier computadora a través de una interfaz común (la clase base **Computadora**), sin necesidad de saber *exactamente* qué tipo específico de computadora es.

Concepto Central: La Interfaz Común

El polimorfismo se basa en la **herencia** y en la **sobrescritura de métodos**.

1. **Clase Base (**Computadora**)**: Define un "contrato" o plantilla. Declara los métodos que *todas* las computadoras deben tener, como **registrar()** o **obtenerEspecificaciones()**.
 2. **Clases Hijas (**Laptop**, **Desktop**)**: Heredan de **Computadora** y *sobrescriben* esos métodos para adaptarlos a su comportamiento específico. Por ejemplo, **registrar()** para una **Laptop** podría incluir el registro de la batería, mientras que para un **Desktop** no.
-

Implementación: Sistema de Registro

Veamos cómo aplicar esto en un sistema que registra computadoras en un inventario.

1. La Clase Base (o Interfaz)

Primero, definimos la plantilla. Usaremos una **clase abstracta** para forzar a las clases hijas a implementar sus propios métodos.

PHP

```
// Clase base abstracta
abstract class Computadora {
    protected string $idSerial;
    protected string $marca;

    public function __construct(string $idSerial, string $marca) {
        $this->idSerial = $idSerial;
        $this->marca = $marca;
    }

    // Método de acceso común
    public function getIdSerial(): string {
```

```

        return $this->idSerial;
    }

    // --- MÉTODOS POLIMÓRFICOS ---
    // Cada clase hija DEBE definir estos métodos

    /**
     * Devuelve una descripción de las especificaciones únicas.
     */
    abstract public function obtenerEspecificaciones(): string;

    /**
     * Ejecuta el proceso de registro específico para este tipo.
     */
    abstract public function registrarEnInventario(): void;
}

```

2. Las Clases Hijas (Concretas)

Ahora, creamos los tipos específicos de computadoras. Observa cómo cada una implementa `obtenerEspecificaciones()` y `registrarEnInventario()` a su manera.

PHP

```

// Clase Hija 1: Laptop
class Laptop extends Computadora {
    protected int $duracionBateriaHoras;

    public function __construct(string $idSerial, string $marca, int $duracionBateria) {
        parent::__construct($idSerial, $marca);
        $this->duracionBateriaHoras = $duracionBateria;
    }

    // Implementación específica
    public function obtenerEspecificaciones(): string {
        return "{$this->marca} Laptop ({$this->idSerial}) - Batería:
{$this->duracionBateriaHoras}hs.";
    }

    public function registrarEnInventario(): void {
        // Lógica específica: registra en la BBDD y también en el sistema de baterías.
        echo "Registrando Laptop {$this->idSerial} (incluyendo módulo de batería)...\n";
    }
}

// Clase Hija 2: Desktop
class Desktop extends Computadora {

```

```

protected bool $tieneFuentePoderATX;

public function __construct(string $idSerial, string $marca, bool $esATX) {
    parent::__construct($idSerial, $marca);
    $this->tieneFuentePoderATX = $esATX;
}

// Implementación específica
public function obtenerEspecificaciones(): string {
    $tipoFuente = $this->tieneFuentePoderATX ? 'ATX' : 'Propietaria';
    return "{$this->marca} Desktop ({$this->idSerial}) - Fuente: {$tipoFuente}.";
}

public function registrarEnInventario(): void {
    // Lógica específica: solo registra en la BBDD principal.
    echo "Registrando Desktop {$this->idSerial}...\n";
}
}

```

3. La Magia del Polimorfismo en Acción ✨

Aquí es donde brilla el concepto. Nuestro sistema de registro no necesita saber si está tratando con una **Laptop** o un **Desktop**. Solo necesita saber que es una **Computadora**.

Crearemos una clase **RegistroInventario** que maneja una lista de computadoras.

```

PHP
class RegistroInventario {
    /**
     * @var Computadora[]
     */
    protected array $computadorasARegistrar = [];

    // El método clave: Acepta CUALQUIER objeto que sea una Computadora.
    public function agregarParaRegistro(Computadora $comp) {
        $this->computadorasARegistrar[] = $comp;
    }

    public function procesarRegistros() {
        echo "--- Iniciando Proceso de Registro de Inventario ---\n";

        // --- ¡AQUÍ ESTÁ EL POLIMORFISMO! ---
        // No necesitamos un if ($comp es Laptop) o if ($comp es Desktop)
        // Simplemente llamamos al método, y PHP (gracias al polimorfismo)
        // sabe qué versión específica del método ejecutar.
    }
}

```

```

        foreach ($this->computadorasARegistrar as $computadora) {
            // 1. Llama al método polimórfico
            $computadora->registrarEnInventario();

            // 2. Llama a otro método polimórfico
            echo "Especificaciones: " . $computadora->obtenerEspecificaciones() . "\n";
            echo "-----\n";
        }
    }
}

```

// --- Uso del Sistema ---

```

// 1. Creamos objetos de diferentes tipos
$miLaptop = new Laptop('SN-LAP-123', 'Dell', 8);
$pcOficina = new Desktop('SN-DESK-456', 'HP', true);
$otraLaptop = new Laptop('SN-LAP-789', 'Apple', 14);

// 2. Creamos el registro
$inventario = new RegistroInventario();

// 3. Agregamos las computadoras. El método agregarParaRegistro()
// trata a todas por igual, como simples 'Computadora'.
$inventario->agregarParaRegistro($miLaptop);
$inventario->agregarParaRegistro($pcOficina);
$inventario->agregarParaRegistro($otraLaptop);

// 4. Procesamos
$inventario->procesarRegistros();

```

Salida del Script:

```

--- Iniciando Proceso de Registro de Inventario ---
Registrando Laptop SN-LAP-123 (incluyendo módulo de batería)...
Especificaciones: Dell Laptop (SN-LAP-123) - Batería: 8hs.
-----
Registrando Desktop SN-DESK-456...
Especificaciones: HP Desktop (SN-DESK-456) - Fuente: ATX.
-----
Registrando Laptop SN-LAP-789 (incluyendo módulo de batería)...
Especificaciones: Apple Laptop (SN-LAP-789) - Batería: 14hs.
-----

```

Resumen del Beneficio

Como puedes ver, el bucle `foreach` en `procesarRegistros` es increíblemente simple. Llama a `$computadora->registrarEnInventario()` en cada objeto.

- Cuando `$computadora` es una `Laptop`, se ejecuta el `registrarEnInventario()` de la clase `Laptop`.
- Cuando `$computadora` es un `Desktop`, se ejecuta el `registrarEnInventario()` de la clase `Desktop`.

Tu clase `RegistroInventario` no necesita cambiar nunca, aunque en el futuro agregues `Servidor`, `Tablet` o `Workstation`. **El polimorfismo te permite extender tu sistema sin modificar el código existente.**